DATA305 / DATA475 · Machine Learning Methods

Sequence Modeling I — Forecasting Setup

Time Series · Windowing · Chronological Split · Baselines · SimpleRNN

Order is a feature — not an accident. Beat the baseline before you cheer.

Lecture 13 Week 10 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda & Learning Objectives

Lecture 13

Five sections · Jena Climate dataset (~70K hourly rows after resample) · univariate first

01 Sequences Are Different

Why order matters; shuffling destroys time-series structure.

02 Jena Climate

Read CSV → resample 10-min to hourly → univariate temperature.

03 Windowing & Split

24 → 1 sliding window; chronological split; data leakage rule.

04 Baselines First

Naive, seasonal-naive, Dense linear; MAE bar chart benchmark.

05 SimpleRNN

Hidden-state intuition, vanishing-gradient teaser, code skeleton.

Learning Objectives

- ✓ Explain why time-series ≠ tabular ≠ image.
- ✓ Build a windowed forecasting pipeline correctly.
- ✓ Compare an RNN against naive baselines using MAE.

Lectures 11–12 → Lecture 13

Bridge

We finished Computer Vision with image segmentation. Now we leave space and enter time.

Lectures 11–12 — Covered

- ✓ Image segmentation: semantic vs instance vs panoptic.
- ✓ U-Net architecture: encoder, skip connections, decoder.
- ✓ SAM zero-shot prototyping with prompts.
- ✓ Pixel-level metrics: IoU, Dice, mIoU.
- ✓ End of the Computer Vision cluster.

Lecture 13 — Today

- → Why time-series differ from tabular and image data.
- → Jena Climate: real meteorological data, hourly temperature.
- → Sliding windows: 24 hours in → 1 hour ahead out.
- → Chronological split: the data-leakage rule for time series.
- → Three baselines before any neural model: MAE benchmark.
- → SimpleRNN — hidden state intuition.

« Order is a feature, not an accident — shuffle a time series and you destroy your dataset. »

01

Sequences Are Different

Why order matters · what shuffling breaks

Three Data Types — Side by Side

Section 01

Tabular and image data are independent samples. Sequences are not.

TABULAR

Housing prices

Each row is independent: one house, one price. No relationship between row 7 and row 8 in the file.

Shuffle is

✓ **FINE — order is irrelevant**

IMAGE

CIFAR-10

Each image has 2-D spatial structure inside, but images are still independent samples — ordering doesn't matter.

Shuffle is

✓ **FINE — samples are independent**

SEQUENCE

Jena temperature

Today's temperature depends on yesterday's. Order IS the information; consecutive rows are tightly correlated.

Shuffle is

✗ **DESTRUCTIVE — breaks the dataset**

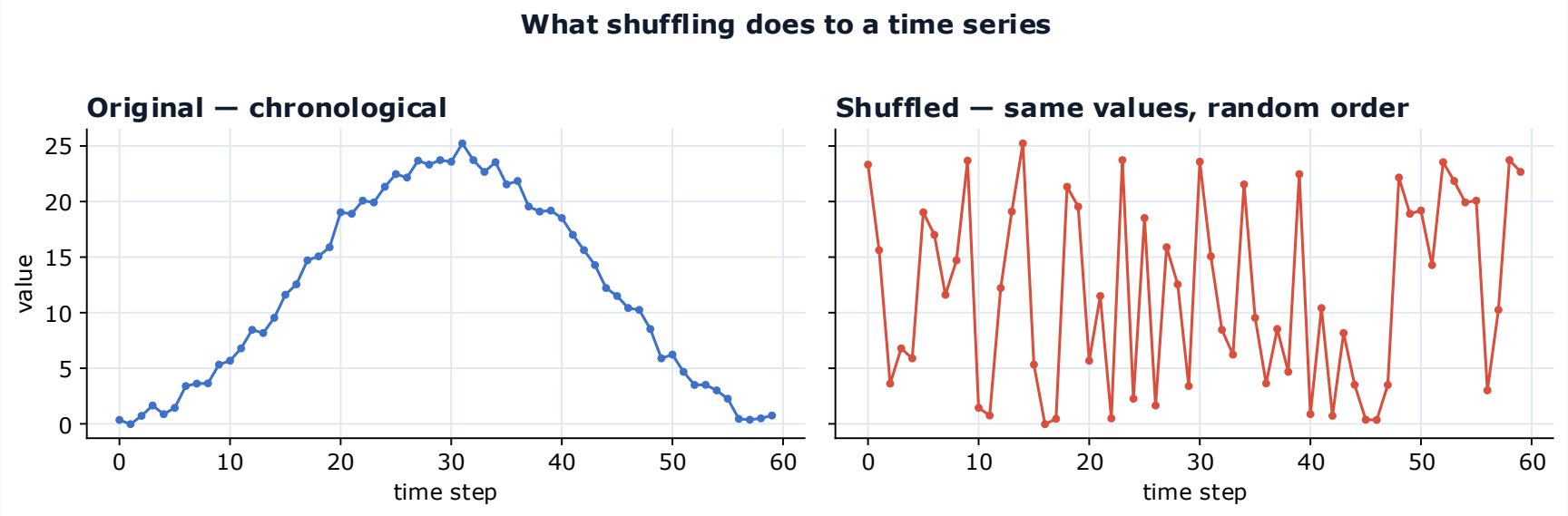
Important: DO NOT shuffle raw time points before splitting/windowing. Shuffling training windows AFTER a chronological split is a separate matter — and is usually fine.

Order is a feature only for sequences — neighbors carry information that random reshuffling erases

What Shuffling Does to a Time Series

Section 01

If you shuffle a time series, distant time points become neighbors.



Same data, same values — but the right plot has no learnable pattern. *A shuffled time series teaches the model nothing about seasonality.*

Synthetic illustration — 60 time steps, same values randomly permuted

02

Jena Climate

From raw 10-min CSV to hourly temperature

Jena Climate — The Dataset

Section 02

Real meteorological data from Jena, Germany — what we actually have on disk.

~420K

Rows in raw CSV

10 min

Sampling interval

14

Meteorological columns

What's in the file

Weather station logs from 2009–2016, sampled every 10 minutes.

Columns include temperature, pressure, humidity, wind speed, dew point, and more.

Loaded with one Keras helper:

```
path = keras.utils.get_file("jena_climate_2009_2016.csv.zip", ...)
```

Today we use only **T (degC)** — univariate first. Multivariate is a tutorial extension.

⚠ **Common misconception:** Jena is not hourly out-of-the-box — the raw CSV is 10-minute. We resample on the next slide.

Source: Max Planck Institute for Biogeochemistry, Jena, Germany · available via tf.keras.utils.get_file()

Preprocessing — Four Steps, None Skippable

Section 02

From raw 10-minute multivariate CSV to a clean hourly univariate signal.

1

Read CSV

pandas.read_csv
420K rows × 15 cols

→

2

Resample

10-min → 1-hour
420K → ~70K rows

→

3

Select column

Keep T (degC) only
Univariate signal

→

4

Plot 2 years

Visualize seasonality
(sanity check)

Why resample to hourly?

- Forecasting one hour ahead is more meaningful for teaching than one 10-minute step.
- 6× fewer rows means faster training in lecture; the seasonality of interest survives.

```
df = pd.read_csv(path)
df['Date Time'] = pd.to_datetime(df['Date Time'], format='%d.%m.%Y %H:%M:%S')
hourly = df.set_index('Date Time').resample('1h').mean(numeric_only=True)
hourly = hourly.ffill().bfill()
```

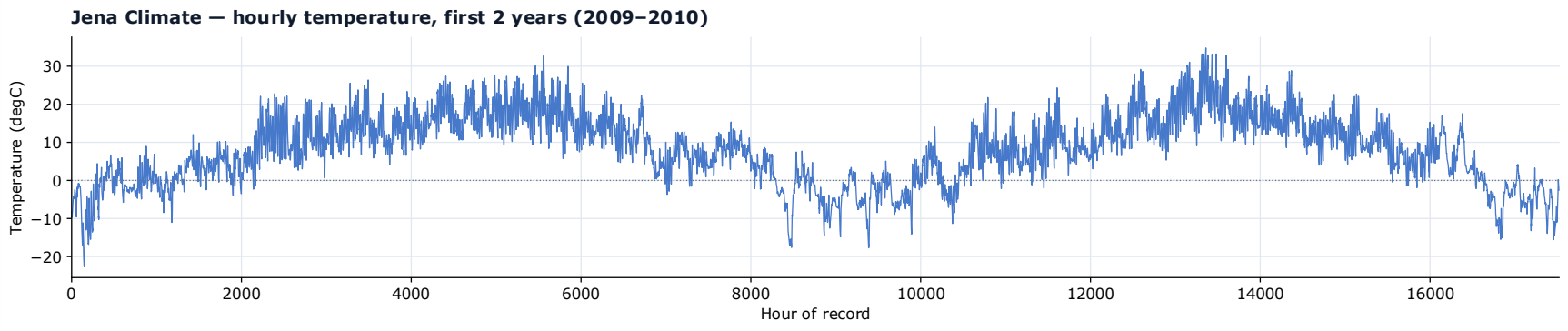
Resampling may create hourly NaNs when no 10-min records exist. ffill() fills them with the last known value, while bfill() handles missing rows at the start.

What the Data Looks Like — Two Scales

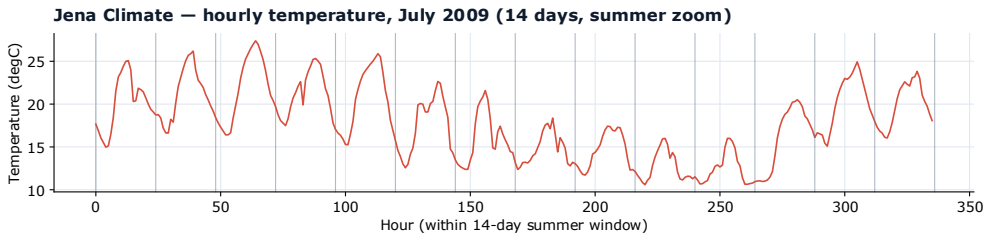
Section 02

Yearly cycle dominates at scale; daily oscillation emerges in the summer zoom.

2009–2010 · yearly seasonality



July 2009 · 14-day summer zoom · daily cycle visible



Yearly cycle dominates. Winter cold, summer warm — amplitude ≈ ±15 °C around the mean.

Daily cycle of ±5 °C exists but is too dense at the 2-year scale — only visible when zoomed.

Weather noise is the residual that no model perfectly forecasts.

Actual Jena Climate 2009–2016 (July 2009 summer zoom)

03

Windowing & Chronological Split

Targets, sliding windows · the data leakage rule

Sliding Window — From a Series to (X, y) Pairs

Section 03

A model wants tensors. We turn the time series into many input/target pairs.

Definition

Window (input)
The last 24 hourly values: 24 floats.

Target
The next hour's temperature: 1 float.

Stride
1 — slide one step at a time.

Result
Many (X, y) pairs; X has shape (24, 1), y is a scalar.

Off-by-one alignment — read carefully

⚠ **Off-by-one:** the target **y[t]** is NOT in the input window. The window stops at **y[t-1]**.

Helper in Keras: tf.keras.utils.timeseries_dataset_from_array — set sequence_length=24, sequence_stride=1

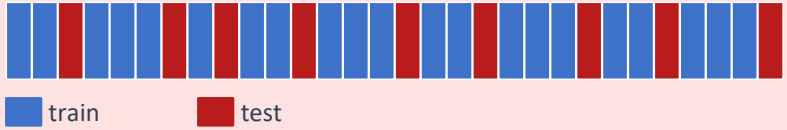
Data Leakage — The One Slide That Saves You

Section 03

Time-series test sets must always be later in time than training sets.

✗ WRONG · Random split

Test rows scattered across all years.



Why it lies: the test sample for Aug 15 sits between train samples for Aug 14 and Aug 16. The model has effectively seen the surrounding context — test accuracy is inflated.

✓ CORRECT · Chronological split

All training data is earlier in time than the test data.

Why it works: the test set is genuinely future data the model never saw. Errors here reflect real forecasting performance.

Iron rule: *for time-series, the test set is always later in time than the train set.*

Same rule applies to financial data, sensor logs, and any sequence with autocorrelation

Normalize — But Fit on Train Only

Section 03

Mean and standard deviation must come from the train slice; never peek at val or test.

✗ WRONG

```
# fits stats on ALL data
mean = data.mean()
std  = data.std()
data = (data - mean) / std
# leaked!
```

✓ CORRECT

```
mean = train.mean()
std  = train.std()
train = (train - mean) / std
val   = (val   - mean) / std
test  = (test  - mean) / std
```

Why this matters

- Split first, then fit preprocessing on the training slice only.
- Same logic applies to imputation, scaling, and any fit/adapt step. Train-only or it is wrong.
- If MAE is reported in °C but targets were standardized, multiply the metric by std before plotting.

« The test set must not leak into your preprocessing pipeline. »

Same principle as L9: tokenizer.adapt(train_only); mean/std fit on train only — fit/adapt is train-only forever

Quick Quiz · 1 of 2

Recap

You have 3 years of daily stock prices. You label each day as **UP or DOWN**, shuffle the rows, do an 80/20 train/test split, train an LSTM classifier, and report **95% test accuracy.**

What is the MOST LIKELY problem with this result?

A An LSTM is too complex for stock data; a simpler model would generalize better.

B Random shuffling caused data leakage — test rows lie next to training rows in time.

C The 80/20 ratio is too generous on training data; 60/40 would have been safer.

D Daily resolution is too coarse; sub-daily data would not produce inflated accuracy.

💡 ***Think about the iron rule we just discussed.***

04

Baselines First

Naive · seasonal-naive · Dense linear · MAE

Why Three Baselines Before a Neural Model

Section 04

You cannot tell whether your RNN is good unless you know what 'good' means.

Naive / Persistence

Definition

$$\hat{y}_t = y_{t-1}$$

Why it exists

Next hour ≈ current hour. A strong parameter-free rule — hard to beat at 1-hour horizon because adjacent temperatures are highly correlated.

Seasonal-naive (lag-24)

Definition

$$\hat{y}_t = y_{t-24}$$

Why it exists

Same hour yesterday. Captures daily periodicity but ignores today's actual weather. Much weaker than persistence for 1-hour-ahead.

Dense linear

Definition

$$\hat{y} = W \cdot x + b$$

Why it exists

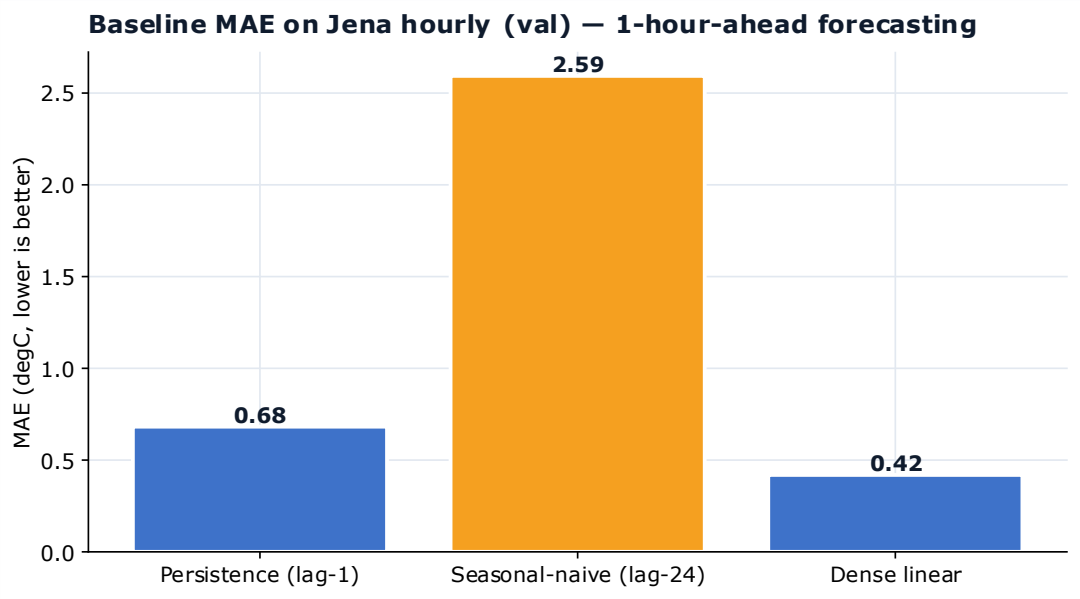
A learned weighted combination of the 24-hour window. Consistently the best simple baseline in this run — the true bar to beat.

These three numbers form your benchmark floor. Every neural model you train this lecture and the next is judged against them. *If your fancy model loses to the best simple baseline, the model is wrong — not the baseline.*

Baseline MAE on Jena Hourly — The Floor

Section 04

Actual Colab output: val MAE in °C. Dense linear is the best simple baseline in this run.



Persistence (lag-1)

0.68 °C — strong rule because adjacent hours are similar.

Seasonal-naive (lag-24)

2.59 °C — captures daily cycle but misses today's state.

Dense linear ← best simple baseline

0.42 °C — learns weights over the 24-hour window. The bar SimpleRNN must beat.

Coming up: we add a SimpleRNN to this same chart. ***Will it beat the best simple baseline?***

Validation MAE from Colab notebook — Jena 2009–2016, 1-hour-ahead univariate

05

SimpleRNN

Hidden state · vanishing gradients · code skeleton

SimpleRNN — Hidden State Intuition

Section 05

Reading a sentence: short-term memory updates with each word + what you remembered before.

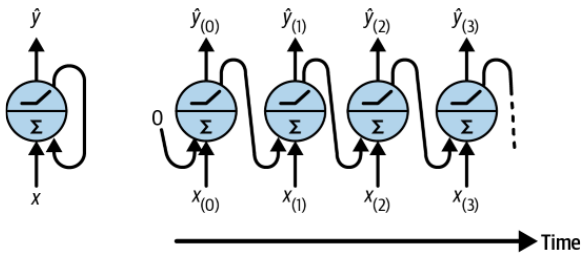
Analogy: reading a sentence

When you read "The temperature was rising before…", your understanding at each word is shaped by:

- the current word you're reading,
- the running understanding from earlier words.

An RNN does the same thing for time steps. The hidden state h_t is the running short-term memory.

Géron Fig. 15-1 — the same neuron unrolled through time




```
h_t  =  f( W_h · h_{t-1}  +  W_x ·
x_t  +  b )
```

One equation applied at every step.

Backprop through time (BPTT) must multiply many copies of W_h together — that multiplicative chain is why gradients vanish.

Géron Fig. 15-1 · left: neuron with recurrent connection; right: unrolled through time steps

Vanishing Gradient — Why SimpleRNN Forgets

Section 05

Repeated multiplication can shrink gradients to zero — and sometimes blow them up.

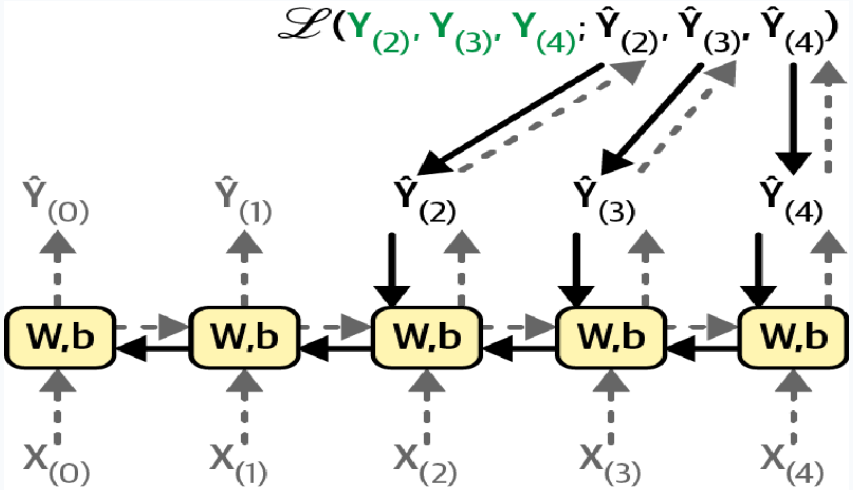
Multiply small numbers, many times

0.9 ^ 10	≈ 0.349
0.9 ^ 50	≈ 0.005
0.9 ^ 100	≈ 0.000027
0.5 ^ 100	≈ 10^{-30}

If repeated factors are mostly below 1, the product shrinks exponentially → vanishing.

If they are mostly above 1, the product blows up → exploding (clipped in practice).

Géron Fig. 15-5 — gradients flow backward through time (BPTT)



BPTT: backward arrows multiply weight matrix repeatedly. Small factors → vanishing; large → exploding. → *motivation for LSTM/GRU in L14.*

SimpleRNN — Code Skeleton & Result

Section 05

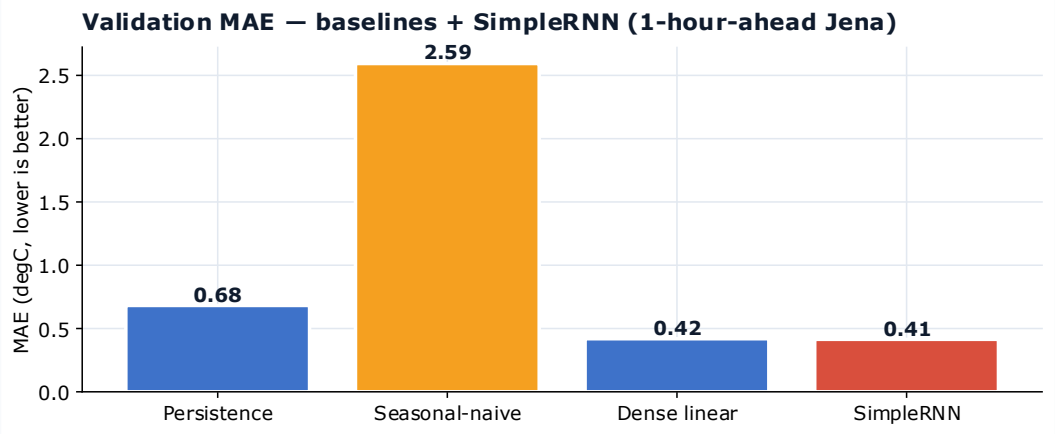
One layer; one Dense head; train it; add it to the bar chart.

```
# 24-hour window → 1 hour ahead
model = tf.keras.Sequential([
    tf.keras.Input(shape=(24, 1)),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1),
])

model.compile(
    optimizer='adam', loss='mse',
    metrics=['mae'])

model.fit(ds_train, epochs=RNN_EPOCHS,
          validation_data=ds_val)
```

Validation MAE — actual Colab output



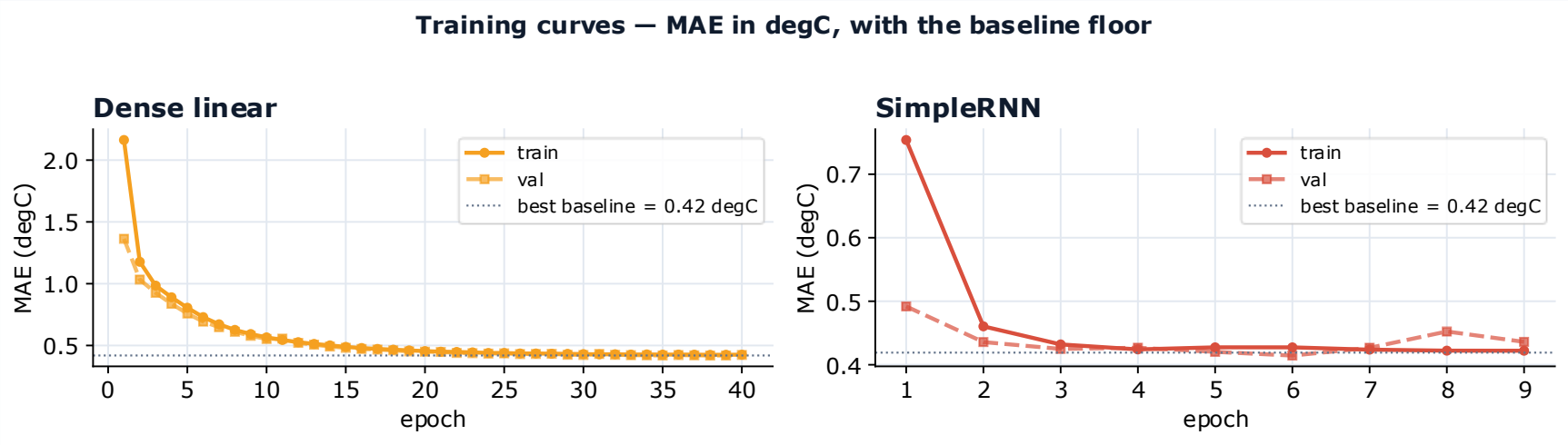
Honest result: SimpleRNN (0.41 °C) slightly beats Dense linear (0.42 °C) — a ~1% MAE reduction. ***The gain is real only because we measured it against the best simple baseline.***

Training Curves & Forecast — Visual Sanity Check

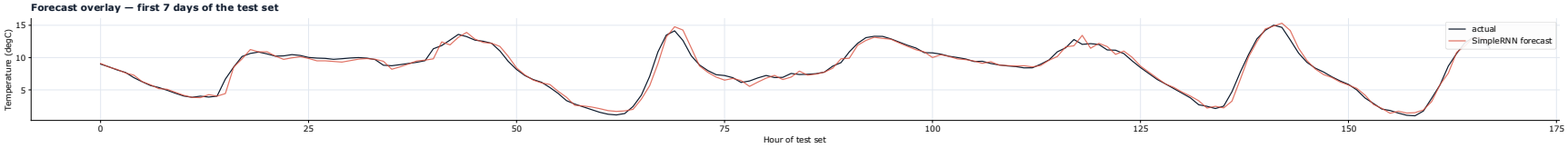
Section 05

Colab output: MAE in °C vs epoch (left), and SimpleRNN forecast vs actual for the first 7 test days (right).

Training curves — Dense linear and SimpleRNN · dashed = validation · dotted = best baseline



Forecast overlay — first 7 test days · navy = actual, red = SimpleRNN



Quick Quiz · 2 of 2

Recap

On Jena hourly, your Colab notebook reports these validation MAEs:

SimpleRNN = 0.414 °C **Dense linear = 0.419 °C** **Persistence = 0.682 °C**

What is the most reasonable conclusion?

A SimpleRNN is dramatically better — baselines are no longer needed for evaluation.

B SimpleRNN slightly beats the best simple baseline, but the gain is small and must be weighed against added complexity.

C The metric is wrong: MAE always favors the simpler model; accuracy should be used instead.

D Seasonal-naive is the correct baseline here because the data has daily cycles.

💡 ***The gain of 0.005 °C is real — but only meaningful because we compared to the best simple baseline.***

Actual numbers from Colab notebook run — Dense linear is the best simple baseline in this experiment

Practical Pitfalls — Checklist

Recap

Time-series mistakes that cost hours in real projects.

01 Random shuffle on a sequence

Always destroys the dataset. Use chronological split — early years train, later years test. No exceptions.

02 Normalize on all data

Fit mean/std on train only. Same for any adapt/scale step. Test data must not influence preprocessing.

03 Off-by-one in target alignment

Window y[t-24:t-1] predicts y[t]. The target is NOT inside the window. Sanity check by printing one (X, y) pair.

04 Skipping simple baselines

Without simple baselines — especially the best simple baseline (Dense linear here) — you cannot tell if your neural model adds value.

05 Multivariate before univariate

Don't open 14 columns. Get the pipeline right with one column first; expand later.

06 Confusing split-shuffle and window-shuffle

Wrong: random-split raw time points across train/val/test. Usually OK: shuffle training windows AFTER a chronological split. Never shuffle val/test.

Summary · Lecture 13

Wrap-up

Get the pipeline right; baseline first; SimpleRNN is the start, not the answer.

01	**Sequences are different**	Order is information; shuffling destroys the dataset. Time-series ≠ tabular ≠ image.
02	**Build the windowed pipeline correctly**	Read CSV, resample, select column, plot. Use sliding windows of 24 → 1; verify alignment.
03	**Split chronologically; normalize on train only**	Iron rule: test set is later in time. Compute mean/std from train; apply to all splits.
04	**Baselines first**	Persistence, seasonal-naive, Dense linear. Compare against simple rules before claiming model success.
05	**SimpleRNN: recurrence helps slightly in this run**	0.005 °C improvement over Dense linear — real but small. Hidden state + vanishing gradient → LSTM/GRU on Friday.

Lecture 14 preview · LSTM and GRU. Gating mechanisms that help recurrent cells remember.